

```
!pip install boto3
```

```
Collecting boto3
  Downloading boto3-1.40.41-py3-none-any.whl.metadata (6.7 kB)
Collecting botocore<1.41.0,>=1.40.41 (from boto3)
  Downloading botocore-1.40.41-py3-none-any.whl.metadata (5.7 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3)
  Downloading jmespath-1.0.1-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.15.0,>=0.14.0 (from boto3)
  Downloading s3transfer-0.14.0-py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1.40.41)
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1.40.41)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<3.0.0,>=2.1->botocore<1.41.0,>=1.40.41)
Downloading boto3-1.40.41-py3-none-any.whl (139 kB)
  139.3/139.3 kB 10.4 MB/s eta 0:00:00
Downloading botocore-1.40.41-py3-none-any.whl (14.0 MB)
  14.0/14.0 MB 111.6 MB/s eta 0:00:00
Downloading jmespath-1.0.1-py3-none-any.whl (20 kB)
Downloading s3transfer-0.14.0-py3-none-any.whl (85 kB)
  85.7/85.7 kB 7.7 MB/s eta 0:00:00
Installing collected packages: jmespath, botocore, s3transfer, boto3
Successfully installed boto3-1.40.41 botocore-1.40.41 jmespath-1.0.1 s3transfer-0.14.0
```

```
import boto3
import json

# Replace with your keys (⚠️ don't share these publicly)
AWS_ACCESS_KEY = "AKIAVSX4Z23PH4R4CRWQ"
AWS_SECRET_KEY = "kIu79SSQ/aw0RL/cTFbhk20cjxjvzLhP8iEzXtmx"
AWS_REGION = "us-east-1" # Use region where Claude 3 Sonnet is available

MODEL_ID = "anthropic.claude-3-sonnet-20240229-v1:0"

# Create boto3 client with credentials
rt = boto3.client(
    "bedrock-runtime",
    region_name=AWS_REGION,
    aws_access_key_id=AWS_ACCESS_KEY,
    aws_secret_access_key=AWS_SECRET_KEY
)

def structured_summary(text, sources):
    body = {
        "anthropic_version": "bedrock-2023-05-31",
        "max_tokens": 500,
        "messages": [
            {
                "role": "user",
                "content": f"""You are a helpful assistant that creates structured summaries.
Perform a Chain-of-Verification and output a structured summary
with main points, supporting evidence, and contradictions.

Text to summarize: {text}
Sources: {sources}"""
            }
        ]
    }

    response = rt.invoke_model(
        modelId=MODEL_ID,
        body=json.dumps(body),
        contentType="application/json"
    )

    result = json.loads(response["body"].read())
    return result["content"][0]["text"]

if __name__ == "__main__":
    input_text = "OpenAI released GPT models that improve reasoning and summarization tasks."
    sources = ["TechCrunch article, March 2024", "Official OpenAI blog post"]

    out = structured_summary(input_text, sources)
    print("\n--- Structured Summary ---\n")
    print(out)
```

```
--- Structured Summary ---
```

Here is a structured summary of the given text, with main points, supporting evidence, and contradictions.

Main Points:

1. OpenAI released new GPT (Generative Pre-trained Trans^{former}) models.

2. The new GPT models are claimed to improve performance on reasoning and summarization tasks.

Supporting Evidence:

1. A TechCrunch article from March 2024 reported on the release of new GPT models by OpenAI.
2. The official OpenAI blog post announced the release of these models and highlighted their improved capabilities in reasoning

Contradictions:

- No contradictory information was provided in the given text or sources.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.